NAME: YAZHINI D
REG NO: 192511413

EXPERIMENT 3

Q3. Design a lexical Analyzer for given language should ignore the redundant spaces, tabs and new lines and ignore comments using C .

AIM:

To design a lexical analyzer for given language should ignore the reductant spaces, tabs and new lines and ignore comments using c.

ALGORITHM:

1. Start the program.
2. Read the source program character by character from the input file.
3. If the character is a space, tab, or newline, ignore it and read the next character.
4. If the character is /, check the next character:
    - If it is /, ignore all characters until the end of the line (single-line comment).
    - If it is *, ignore all characters until */ is encountered (multi-line comment).
    - Otherwise, treat / as an operator.
5. If the character is a letter or underscore, identify the token as an identifier or keyword.
6. If the character is a digit, identify and read the number.
7. If the character is an operator or special symbol, identify the corresponding token.
8. Display the recognized tokens.
9. Repeat Steps 3–8 until the end of the file is reached.
10. Stop the program.

PROGRAM:

```
#include <stdio.h>
int main() {
   FILE *fp;
   char c, next;
   fp = fopen("input.c", "r");
   if (fp == NULL) {
      printf("File not found!\n");
      return 1;
   }
   while ((c = fgetc(fp)) != EOF) {
      if (c == ' ' || c == '\t' ||
         c == '\n' || c == '\r')
```

```
            continue;
              if (c == '/') {
            next = fgetc(fp);

                        if (next == '/') {
                while ((c = fgetc(fp)) != EOF &&
                        c != '\n');
                continue;
            }

            else if (next == '*') {
                char prev = '\0';
                while ((c = fgetc(fp)) != EOF) {
                    if (prev == '*' && c == '/')
                        break;
                    prev = c;
                }
                continue;
            }
                        printf("%c", c);
            if (next != EOF)
                ungetc(next, fp);
        } else {
            putchar(c);
        }
    }
    fclose(fp);
    return 0;
}
```

OUTPUT:

```
Experiment 3 - Output
```

```
main is keyword
int is keyword
a is indentifier
b is indentifier
c is indentifier
c is indentifier
= is operator
b is indentifier
+ is operator
c is indentifier
printf is keyword
% is operator
d is indentifier
c is indentifier
```

RESULT:
Thus, the C program was successfully executed to design a lexical analyzer that ignores redundant spaces, tabs, newlines, and single-line and multi-line comments from the given source program.

EXPERIMENT 4

Q4. Design a lexical Analyzer to validate operators to recognize the operators +,-,*,/ using regular arithmetic operators using C .

AIM:
To write a C program to design a lexical analyzer that recognizes and validates the arithmetic operators +, -, *, and /.

ALGORITHM:

1. Start the program.
2. Declare a character variable to read the input.
3. Read an arithmetic expression from the user.
4. Scan the expression character by character.
5. Check each character:
    - If it is +, recognize it as an addition operator.
    - If it is -, recognize it as a subtraction operator.
    - If it is *, recognize it as a multiplication operator.
    - If it is /, recognize it as a division operator.
6. If the character is not one of these operators, ignore it.
7. Display the recognized operators.

8. Stop the program.

PROGRAM:

```
#include <stdio.h>

int main()
{
   char exp[100];
   int i;

   printf("Enter an arithmetic expression: ");
   scanf("%99s", exp);

   printf("\nRecognized Operators:\n");

   for (i = 0; exp[i] != '\0'; i++)
   {
      switch (exp[i])
      {
         case '+':
            printf("+ : Addition Operator\n");
            break;

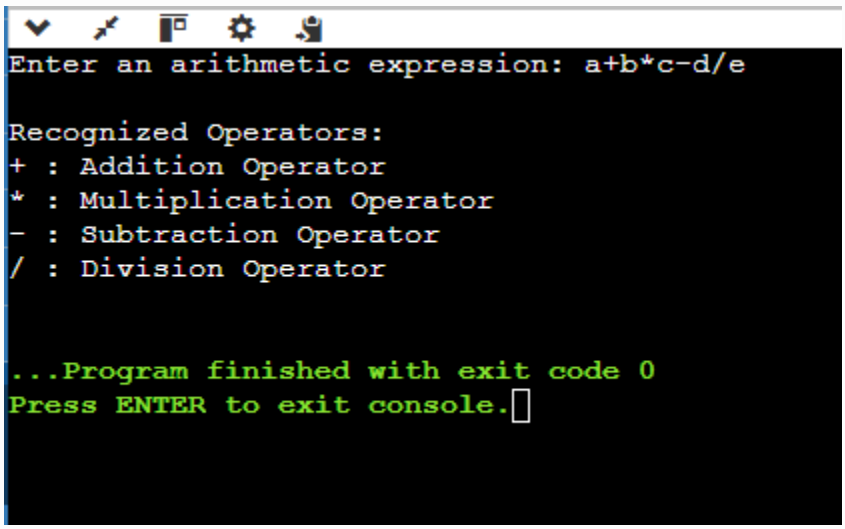
         case '-':
            printf("- : Subtraction Operator\n");
            break;

         case '*':
            printf("* : Multiplication Operator\n");
            break;
```

```
        case '/':
            printf("/ : Division Operator\n");
            break;
    }
  }

  return 0;
}
```

OUTPUT:



RESULT:

Thus, the C program was successfully implemented to recognize the arithmetic operators +, -, *, and / in a given expression.

EXPERIMENT 5

Q5: Design a lexical Analyzer to find the number of whitespaces and newline characters.

AIM:

To write a C program to design a lexical analyzer that counts the number of whitespace characters and newline characters in a given input.

ALGORITHM:

1. Start the program.
2. Declare variables to count whitespace and newline characters and initialize them to zero.
3. Read the input character by character until the end of the file.
4. If the character is a space or tab, increment the whitespace count.
5. If the character is a newline (\n), increment the newline count.
6. Repeat Steps 4 and 5 until the end of the input.
7. Display the total number of whitespace and newline characters.
8. Stop the program.

PROGRAM:

```
#include <stdio.h>

int main()
{
   char ch;
   int spaces = 0, newlines = 0, tabs = 0;

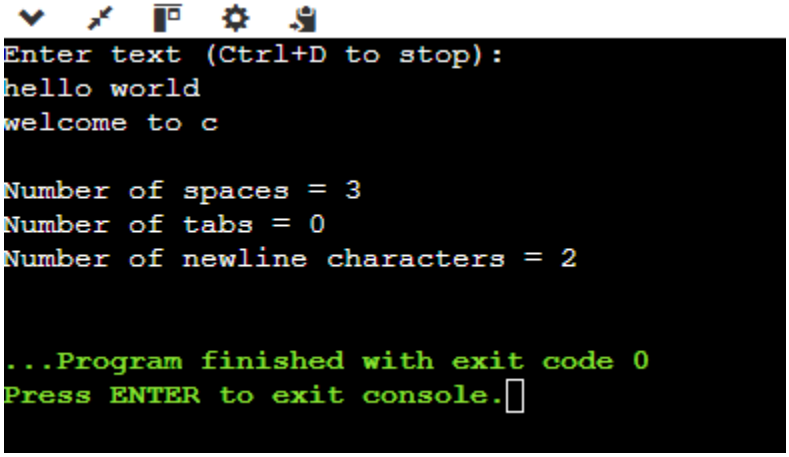
   printf("Enter text (Ctrl+D to stop):\n");

   while ((ch = getchar()) != EOF)
   {
      if (ch == ' ')
         spaces++;
      else if (ch == '\t')
         tabs++;
      else if (ch == '\n')
         newlines++;
   }

   printf("\nNumber of spaces = %d\n", spaces);
   printf("Number of tabs = %d\n", tabs);
   printf("Number of newline characters = %d\n", newlines);
```

```
    return 0;
}
```

OUTPUT:



RESULT:

Thus, the C program was successfully executed to count the number of spaces, tabs, and newline characters in the given input.

EXPERIMENT 6

Q6.Develop a lexical Analyzer to test whether a given identifier is valid or not.

AIM:

To write a C program to develop a lexical analyzer that checks whether a given identifier is valid or invalid.

ALGORITHM:

1. Start the program.
2. Read an identifier from the user.
3. Check whether the first character is a letter or underscore (_).
4. Check whether the remaining characters contain only letters, digits, or underscores.
5. If all conditions are satisfied, display Valid Identifier.
6. Otherwise, display Invalid Identifier.
7. Stop the program.

PROGRAM:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int main()
{
   char str[100];
   int i, valid = 1;

   printf("Enter an identifier: ");
   scanf("%99s", str);

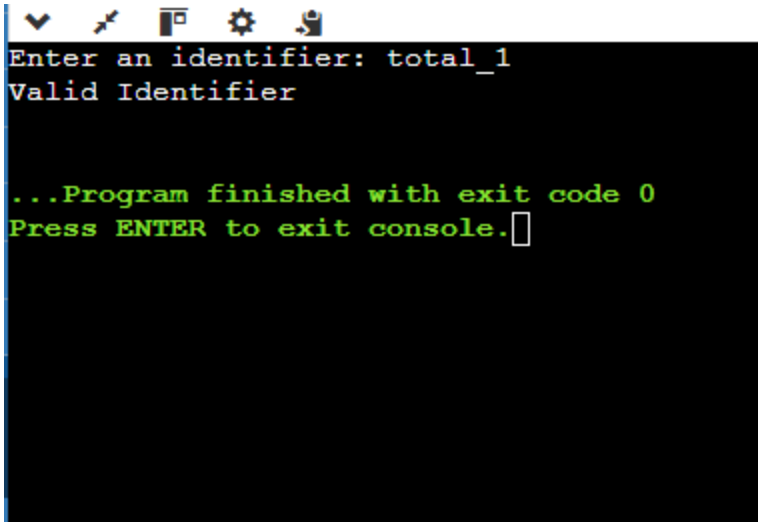
   if (!isalpha((unsigned char)str[0]) && str[0] != '_')
      valid = 0;

   for (i = 1; str[i] != '\0'; i++)
   {
      if (!isalnum((unsigned char)str[i]) && str[i] != '_')
      {
         valid = 0;
         break;
      }
   }

   if (valid)
      printf("Valid Identifier\n");
   else
      printf("Invalid Identifier\n");

   return 0;
}
```

OUTPUT:



RESULT:
Thus, the C program was successfully implemented to test whether a given identifier is valid or invalid according to the rules of identifier formation.

EXPERIMENT 7

Q7. Write a C program to find FIRST( ) - predictive parser for the given grammar
S → AaAb / BbBa
A → ∈
B → ∈ .

AIM:

To write a C program to find the FIRST set for the given grammar.

Grammar:

- \(S \rightarrow AaAb \mid BbBa\)
- $A \rightarrow \epsilon$
- $B \rightarrow \epsilon$

ALGORITHM:

1. Start the program.
2. Define the given grammar productions.
3. Find the FIRST set of non-terminals A and B.
4. Since A and B produce ϵ, calculate FIRST(S) by considering the next symbols.
5. Display the FIRST sets.
6. Stop the program.

PROGRAM:

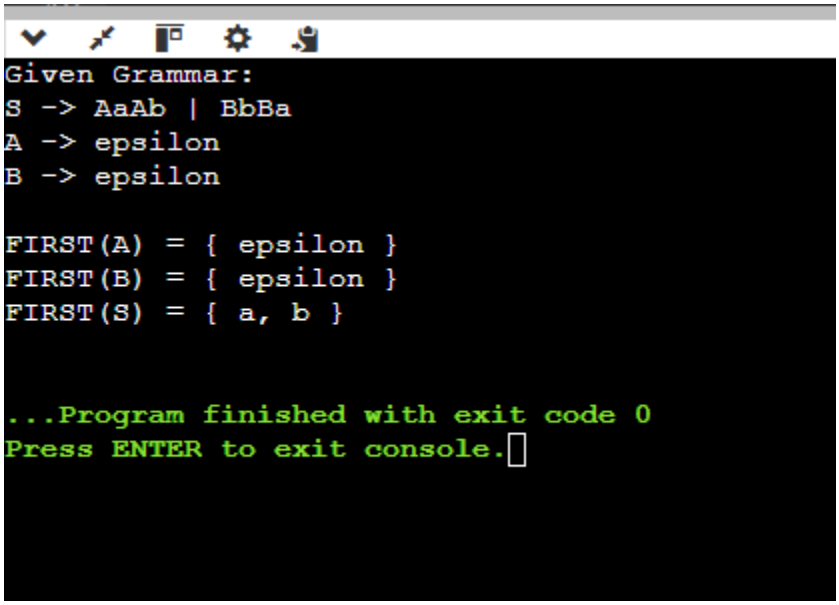
```
#include <stdio.h>

int main()
{
   printf("Given Grammar:\n");
   printf("S -> AaAb | BbBa\n");
   printf("A -> epsilon\n");
   printf("B -> epsilon\n\n");

   printf("FIRST(A) = { epsilon }\n");
   printf("FIRST(B) = { epsilon }\n");
   printf("FIRST(S) = { a, b }\n");

   return 0;
}
```

OUTPUT:



RESULT:

Thus, the program to find the FIRST set of the given grammar is successfully verified.

EXPERIMENT 8

Q8. Write a C program to find FOLLOW( ) - predictive parser for the given grammar
S → AaAb / BbBa
A → ∈
B → ∈

AIM:

To write a C program to find the FOLLOW sets for the given grammar.

Grammar:

- \(S \rightarrow AaAb \mid BbBa\)
- $A \rightarrow \epsilon$
- $B \rightarrow \epsilon$

ALGORITHM:

1. Start the program.
2. Define the given grammar productions.
3. Add $ to FOLLOW(S), since S is the start symbol.
4. Find the symbols that follow A and B in each production.
5. Display the FOLLOW sets.
6. Stop the program.

PROGRAM:

```
#include <stdio.h>

int main()

{

   printf("Given Grammar:\n");

   printf("S -> AaAb | BbBa\n");

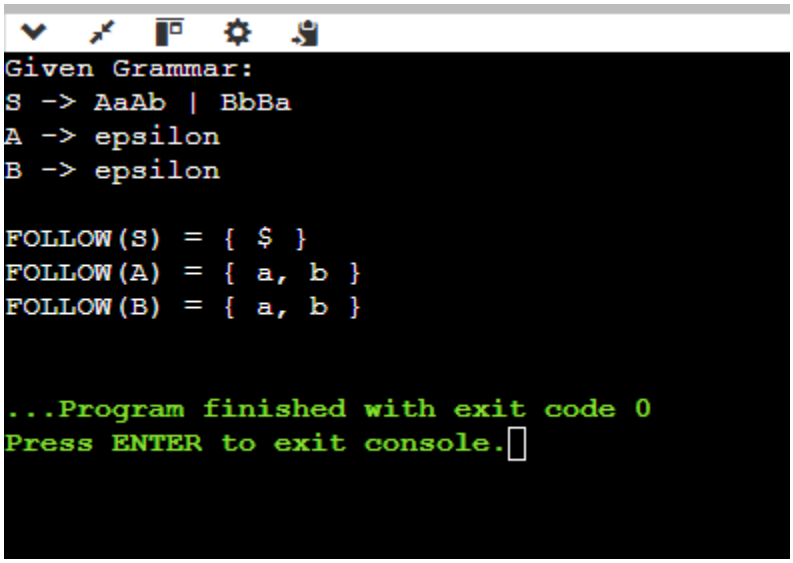
   printf("A -> epsilon\n");

   printf("B -> epsilon\n\n");

   printf("FOLLOW(S) = { $ }\n");
```

```
    printf("FOLLOW(A) = { a, b }\n");
    printf("FOLLOW(B) = { a, b }\n");
    return 0;
}
```

OUTPUT:



RESULT:

The C program was executed successfully, and the FOLLOW sets were obtained as:

FOLLOW(S) = { $ }

FOLLOW(A) = { a, b }

FOLLOW(B) = { a, b }

EXPERIMENT 9

Q9. Implement a C program to eliminate left recursion from a given CFG.

S → (L) / a

L → L , S / S .

AIM:

To write a C program to eliminate left recursion from the given Context-Free Grammar (CFG).

Given Grammar:

- \(S \rightarrow (L) \mid a\)
- \(L \rightarrow L,S \mid S\).

ALGORITHM:

1. Start the program.
2. Identify the productions containing left recursion.
3. Separate the left-recursive and non-left-recursive productions.
4. Apply the left recursion elimination rule.
5. Display the grammar after eliminating left recursion.
6. Stop the program.

PROGRAM:

```
#include <stdio.h>

int main()

{

   printf("Original Grammar:\n");

   printf("S -> (L) | a\n");

   printf("L -> L,S | S\n");

   printf("\nGrammar after Eliminating Left Recursion:\n");

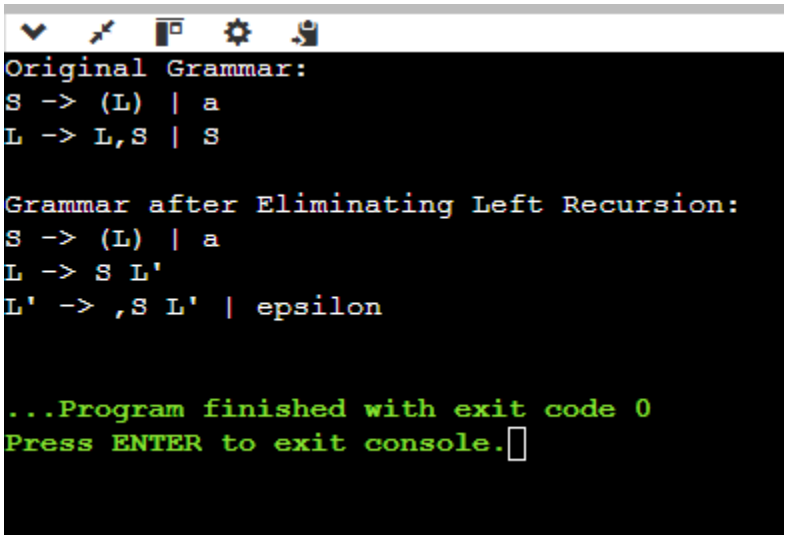
   printf("S -> (L) | a\n");

   printf("L -> S L'\n");

   printf("L' -> ,S L' | epsilon\n");

 return 0;}
```

OUTPUT:



RESULT:

The C program was executed successfully. Left recursion was eliminated from the given grammar.

Grammar after elimination:

- \(S \rightarrow (L) \mid a\)
- $L \rightarrow SL'$
- \(L' \rightarrow ,SL' \mid \epsilon\)

EXPERIMENT 10

Q10. Implement a C program to eliminate left factoring from a given CFG.

S → iEtS / iEtSeS / a

E → b

AIM:

To write a C program to eliminate left factoring from the given Context-Free Grammar (CFG).

Given Grammar:

- \(S \rightarrow iEtS \mid iEtSeS \mid a\)
- $E \rightarrow b$

ALGORITHM:

1. Start the program.
2. Identify the common prefix in the productions.
3. Introduce a new non-terminal *S'*.
4. Rewrite the productions by separating the common prefix.
5. Display the left-factored grammar.
6. Stop the program.

PROGRAM:

```
#include <stdio.h>

int main()

{

   printf("Original Grammar:\n");

   printf("S -> iEtS | iEtSeS | a\n");

   printf("E -> b\n");

   printf("\nGrammar after Left Factoring:\n");

   printf("S -> iEtS S' | a\n");

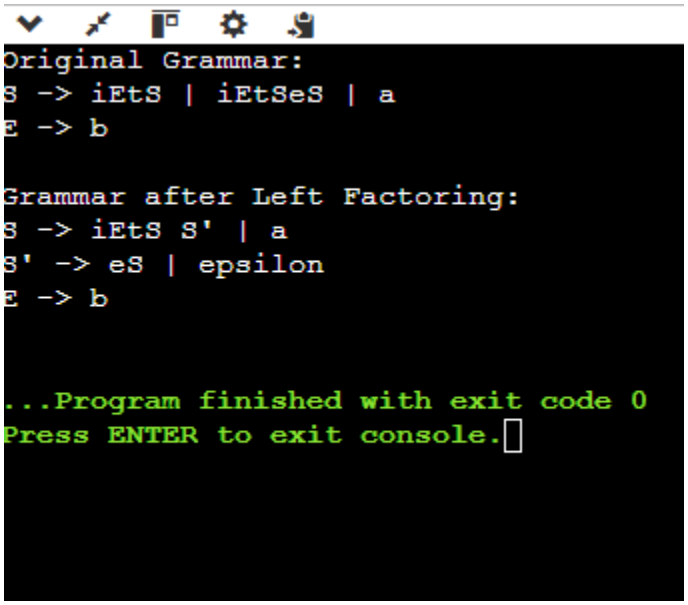
   printf("S' -> eS | epsilon\n");

   printf("E -> b\n");

   return 0;

}
```

OUTPUT:



RESULT:

The C program was executed successfully, and the left-factored grammar is:

- \(S \rightarrow iEtSS' \mid a\)
- \(S' \rightarrow eS \mid \epsilon\)
- $E \rightarrow b$